



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Requirements Specification (SRS)

AttendEase – Student Attendance Management System Team 3 | Course UE24CS341A | PES University

Revision History

Version	Date	Description	Author(s)
1.0	02 September 2026	Initial draft.	Team 3
1.1	03 September 2026	Regrouped System Features into 4 team ownership roles; added FR-037/FR-038.	Team 3
1.2	03 October 2026	Added UML Use Case diagram (Fig. 1) to Section 4.1; split Section 6.3 into Security Objectives and Security Requirements.	Team 3

Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of AttendEase, a web-based Student Attendance Management System developed by Team 3 for the PES University Software Engineering Level 3 Mini-Project (Course UE24CS341A). It is intended to serve as the authoritative reference for design, implementation, and testing of the system.

1.2 Intended Audience

- Team 3 developers implementing AttendEase.
- Course instructor and evaluators assessing the mini-project deliverables.
- Future maintainers or reviewers of the system.

1.3 Product Scope

AttendEase allows administrators to configure courses, sections and faculty assignments; faculty to record attendance per session and manage correction requests; and students to view their attendance and raise correction requests. The system targets a single-department academic deployment.

1.4 References

- PES University SE Level 3 Mini-Project Guidelines and Deliverables Checklist.
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
- AttendEase Project Synopsis, Team 3.
- Coffee Kiosk Example — SE Level 3 Project Submission Guide (reference exemplar).

2. Overall Description

2.1 Product Perspective

AttendEase is a new, self-contained web application. It is not a modification of an existing system. It integrates with an SMTP email service for correction-request notifications but otherwise operates as a standalone system.

2.2 Product Functions

- User registration, login and role-based access control.
- Course, section and timetable management.
- Faculty-to-section assignment.
- Attendance session creation and per-student attendance marking.
- Student attendance dashboard.
- Attendance correction request submission and approval workflow.
- Attendance shortage / defaulter identification.
- Attendance report generation.

2.3 User Classes and Characteristics

User Class	Characteristics
Student	Views own attendance; raises correction requests. Assumed basic web-literacy.

User Class	Characteristics
Faculty	Marks attendance; reviews and approves/rejects correction requests; generates reports for their sections.
Administrator	Configures courses, sections, and faculty assignments; has system-wide visibility.

2.4 Operating Environment

- Server: Python 3.11 / Django 4.x application server.
- Database: PostgreSQL 14+.
- Frontend: Modern web browsers (Chrome, Firefox), Bootstrap 5 UI.
- Deployment: Docker containers, GitHub Actions CI/CD.

2.5 Design and Implementation Constraints

- Must be implemented in Python/Django within one academic semester.
- Must use Git/GitHub with mandatory pull-request-based code review.
- Must support role-based access for exactly three user classes (Student, Faculty, Administrator).
- Must be deployable via Docker for grading/demo purposes.
- Team of 4 members, each owning one of four defined roles.

2.6 Assumptions and Dependencies

- Each student and faculty member has a unique, pre-verifiable email address for account registration.
- Attendance is recorded per scheduled session, not continuously.
- The institution's academic calendar (courses, sections) is set up by an administrator before each semester.
- An SMTP relay is available for correction-request notification emails.
- The system is deployed for a single department/institution instance.

3. External Interface Requirements

3.1 User Interfaces

Three role-specific web interfaces (Student, Faculty, Administrator), built with Django templates and Bootstrap 5, accessible via standard web browsers.

3.2 Software Interfaces

PostgreSQL database via Django ORM; SMTP service via Django's email backend for correction-request notifications.

3.3 Communications Interfaces

HTTPS for all client-server communication; RESTful JSON API between the frontend and backend.

4. Analysis Models

4.1 Use Cases

ID	Use Case	Primary Actor(s)
UC-01	Register / Login	Student, Faculty, Administrator
UC-02	Manage Course & Timetable	Administrator
UC-03	Assign Faculty to Subject-Section	Administrator
UC-04	Start & Mark Attendance Session	Faculty
UC-05	View Attendance Dashboard	Student
UC-06	Raise Correction Request	Student
UC-07	Approve/Reject Correction Request	Faculty
UC-08	Generate Defaulter/Attendance Report	Faculty, Administrator

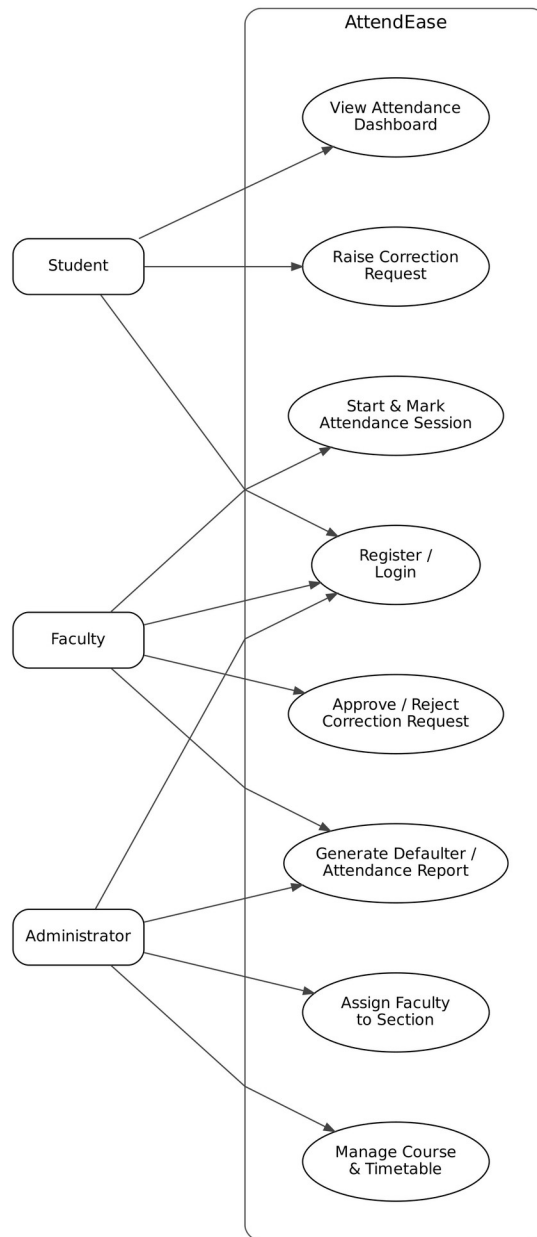


Figure 1: UML Use Case diagram for AttendEase, showing Student, Faculty and Administrator actors against use cases UC-01–UC-08.

4.2 Domain / Entity Model

The core domain entities are User (with role Student/Faculty/Administrator), Course, Section, Timetable, AttendanceSession, AttendanceRecord, and CorrectionRequest. A User with role Faculty is assigned to one or more Sections; each AttendanceSession belongs to a Section and produces one AttendanceRecord per enrolled Student; a CorrectionRequest references exactly one AttendanceRecord and transitions through Pending → Approved/Rejected states.

5. System Features

5.1 Role 1 — User Access & Student Portal

5.1.1 Authentication & Role-Based Access

Req ID	Requirement	Priority
FR-001	The system shall allow a new user to register with name, email, and role.	High
FR-002	The system shall verify email uniqueness at registration.	High
FR-003	The system shall allow a registered user to log in with email and password.	High
FR-004	The system shall enforce role-based access control for every protected endpoint.	High
FR-005	The system shall allow a user to log out, invalidating their session/token.	Medium
FR-034	The system shall allow a user to reset a forgotten password via email verification.	Medium

5.1.2 Student Attendance Dashboard

Req ID	Requirement	Priority
FR-020	The system shall display a student's attendance percentage per enrolled course.	High
FR-021	The system shall display a session-wise attendance history for a student.	Medium
FR-022	The system shall visually flag courses where attendance is below the shortage threshold.	Medium
FR-023	The system shall allow a student to filter attendance history by date range.	Low

5.2 Role 2 — Academic & Attendance Setup

5.2.1 Student & Academic Management

Req ID	Requirement	Priority
FR-006	The system shall allow an administrator to create and edit courses.	High
FR-007	The system shall allow an administrator to create and edit sections within a course.	High
FR-008	The system shall allow an administrator to enroll students into a section.	High
FR-009	The system shall allow an	Medium

Req ID	Requirement	Priority
	administrator to define a weekly timetable per section.	
FR-010	The system shall allow bulk student enrollment via CSV upload.	Low
FR-011	The system shall prevent duplicate section codes within a course.	Medium
FR-012	The system shall allow an administrator to deactivate a course or section.	Low
FR-013	The system shall allow an administrator to assign a faculty member to a section.	High

5.2.2 Attendance Recording

Req ID	Requirement	Priority
FR-014	The system shall allow faculty to start an attendance session for a scheduled section period.	High
FR-015	The system shall display the enrolled-student roster for the active session.	High
FR-016	The system shall allow faculty to mark each student present or absent.	High
FR-017	The system shall allow faculty to bulk-mark all students present with a single action.	Medium
FR-018	The system shall allow faculty to close/finalize a session after marking.	Medium
FR-019	The system shall timestamp each attendance session on creation and closure.	Low

5.3 Role 3 — Attendance Updates & Corrections

5.3.1 Attendance Management

Req ID	Requirement	Priority
FR-037	The system shall prevent duplicate attendance records for the same student and session.	High
FR-038	The system shall allow faculty to edit an attendance record within a configurable edit window after session closure.	Medium

5.3.2 Attendance Correction Requests

Req ID	Requirement	Priority
FR-024	The system shall allow a student to raise a correction request against a	High

Req ID	Requirement	Priority
	specific attendance record, with a reason.	
FR-025	The system shall notify the relevant faculty member by email when a correction request is raised.	Medium
FR-026	The system shall display a queue of pending correction requests to the relevant faculty member.	High
FR-027	The system shall allow faculty to approve a correction request, updating the underlying attendance record.	High
FR-028	The system shall allow faculty to reject a correction request with a reason.	Medium

5.4 Role 4 — Attendance Analytics & Reporting

5.4.1 Attendance Shortage Identification

Req ID	Requirement	Priority
FR-029	The system shall compute each student's attendance percentage per course.	High
FR-030	The system shall identify students below a configurable attendance-shortage threshold.	High
FR-035	The system shall allow the shortage threshold to be configured per course by an administrator.	Medium

5.4.2 Attendance Reports

Req ID	Requirement	Priority
FR-031	The system shall generate a defaulter report listing students below the shortage threshold for a section.	High
FR-032	The system shall generate a section-wise attendance summary report.	Medium
FR-033	The system shall allow reports to be exported as CSV/PDF.	Medium
FR-036	The system shall allow faculty/administrators to filter reports by date range and section.	Low

6. Other Nonfunctional Requirements

6.1 Performance

Req ID	Requirement
NFR-001	95% of API requests shall complete within 2 seconds under

Req ID	Requirement
	expected concurrent load (up to 50 simultaneous users).
NFR-002	The attendance-marking roster shall load within 3 seconds for a section of up to 80 students.

6.2 Safety

Req ID	Requirement
NFR-003	The system shall not permanently delete attendance records; corrections shall be tracked as an auditable change, not a destructive overwrite.
NFR-004	The system shall validate all session/date inputs to prevent attendance being recorded for future dates.

6.3 Security

6.3.1 Security Objectives

The following objectives guide the security design of AttendEase (elaborated further in the Software Architecture & Design Specification, Section 3.9):

Obj ID	Security Objective
SEC-OBJ-01	Ensure that only authenticated users with the correct role can access or modify attendance data (confidentiality and access control).
SEC-OBJ-02	Ensure that every change to an attendance record is attributable and auditable, so that correction-request decisions cannot be repudiated (integrity and accountability).
SEC-OBJ-03	Protect user credentials and session tokens in storage and in transit against disclosure or tampering (data protection).

6.3.2 Security Requirements

Req ID	Requirement
SEC-001	The system shall store all passwords using a salted, one-way hash (never in plaintext).
SEC-002	The system shall lock out or throttle login attempts after repeated failures on an account.
SEC-003	The system shall enforce server-side role-based authorization checks on every write endpoint, independent of client-side UI restrictions.
SEC-004	The system shall scope all attendance queries to the authenticated user's own role and enrolled/assigned sections, preventing cross-section data access.
SEC-005	The system shall require all client-server communication to occur over HTTPS.
SEC-006	The system shall record an audit trail (acted_by, acted_at) for every correction-request approval or rejection.

6.4 Software Quality Attributes

Req ID	Requirement
NFR-005	The system shall be maintainable: each of the four role-owned modules shall be independently testable.
NFR-006	The system shall achieve at least 70% automated test coverage on service-layer business logic by final submission.
NFR-007	The system's UI shall be usable on both desktop and tablet screen widths.
NFR-008	The system shall be deployable via a single docker-compose command for grading/demo purposes.

6.5 Business Rules / Domain Rules

- A student may raise at most one open correction request per attendance record at a time.
- An attendance session, once closed, may only be edited within the configurable edit window (FR-038).
- A faculty member may only mark attendance for sections they are assigned to.
- The attendance-shortage threshold defaults to 75% unless overridden per course by an administrator.
- A correction request, once Approved or Rejected, is immutable.
- Only an administrator may deactivate a course or section.

7. Other Requirements

- The system's database schema shall support future extension to multiple departments without a breaking migration.
- The system shall comply with the institution's data-retention and academic-record-keeping policies.
- Where feasible, the system shall reuse existing open-source Django packages (e.g., djangorestframework-simplejwt) rather than custom-building standard functionality.

Appendix A: Glossary

Term	Definition
Section	A specific offering of a course, with its own enrolled students and assigned faculty.
Attendance Session	A single instance of attendance-taking for one section, on one date/period.
Defaulter	A student whose attendance percentage is below the configured shortage threshold.
Correction Request	A student-raised request to amend an attendance record.
RBAC	Role-Based Access Control.
RTM	Requirements Traceability Matrix.
FR / NFR / SEC	Functional Requirement / Non-Functional Requirement / Security Requirement identifiers used throughout this document.

Appendix B: Field Layouts

B.1 Core Data Fields

Entity	Field
User	id, name, email, password_hash, role
Course	id, code, title
Section	id, course_id, code, faculty_id
Timetable	id, section_id, day_of_week, start_time, end_time
AttendanceSession	id, section_id, date, status, created_at, closed_at
AttendanceRecord	id, session_id, student_id, status
CorrectionRequest	id, record_id, student_id, reason, status, acted_by, acted_at
Enrollment	id, section_id, student_id
Report	id, section_id, generated_by, generated_at, type
Config (threshold)	id, course_id, shortage_threshold
AuditLog	id, entity, entity_id, action, actor_id, timestamp
Notification	id, recipient_id, type, sent_at
PasswordResetToken	id, user_id, token, expires_at
Session Token	id, user_id, token, issued_at, expires_at

B.2 Sample Report Requirements

Report	Required Columns
Defaulter Report	Student Name, USN, Course, Attendance %, Threshold, Status
Section Attendance Summary	Section, Date Range, Average Attendance %, Total Sessions

Appendix C: Requirement Traceability Matrix

A representative sample of the Requirement Traceability Matrix is shown below; the full matrix is maintained alongside the codebase and is updated as architecture, design, and test artifacts (see the AttendEase SAD and Software Test Plan) are completed.

Req ID	Architecture	Design	Code	Test Case
FR-001	Auth API/Service (SAD 3.4)	—	TBD	TC-01
SEC-002	Auth API/Service (SAD 3.4, 3.9)	—	TBD	TC-02
FR-013	Academic Setup API/Service (SAD 3.4)	—	TBD	TC-03
FR-014, FR-017	Attendance API/Service (SAD 3.4)	SAD Fig. 2	TBD	TC-04
FR-037	Correction	SAD Fig. 2	TBD	TC-05

Req ID	Architecture	Design	Code	Test Case
	API/Service (SAD 3.4, 3.8); enforced at runtime by Attendance Service			
FR-020	Student Web UI	—	TBD	TC-06
FR-024	Correction API/Service (SAD 3.4)	SAD Fig. 3	TBD	TC-07
FR-027, SEC-006	Correction API/Service (SAD 3.4)	SAD Fig. 3	TBD	TC-08
SEC-003	Cross-cutting RBAC enforcement (SAD 3.8, 3.9)	—	TBD	TC-09
FR-031	Reporting API/Service (SAD 3.4)	—	TBD	TC-10
NFR-001, NFR-002	Attendance API/Service (SAD 3.4)	—	TBD	TC-11
SEC-004	Attendance API/Service (SAD 3.4, 3.8)	—	TBD	TC-13



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Architecture & Design Specification (SAD)

AttendEase – Student Attendance Management System Team 3 | Course UE24CS341A | PES University

Revision History

Version	Date	Description	Author(s)
1.0	03 October 2026	Initial SAD covering architecture and design for AttendEase.	Team 3
1.1	03 October 2026	Corrected FR-037/038 ownership, SEC/NFR traceability, security-architecture wording, API paths, and Figures 2-3 to match the SRS and the sequence diagrams.	Team 3

Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026

1. Introduction

1.1 Purpose

This Software Architecture & Design Specification (SAD) describes the system architecture and detailed design of AttendEase, a web-based Student Attendance Management System. It translates the functional and non-functional requirements captured in the AttendEase Software Requirements Specification (SRS) into an architectural structure, component design, and interaction design that the development team will implement. This document is intended to guide implementation, support code review, and serve as a reference for testing and maintenance.

1.2 Scope

This SAD covers the architecture of the AttendEase web application, including its layered component structure, the technology stack, the chosen architectural pattern, security architecture, and the detailed design of two representative workflows (attendance marking and correction-request approval) via UML sequence diagrams. It also documents the REST API design, error-handling strategy, and traceability between architectural components and the requirements defined in the SRS.

1.3 Intended Audience

- Team 3 developers implementing AttendEase modules.
- Course instructor and evaluators assessing the Level 3 mini-project.
- Future maintainers extending or reviewing the system.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
SAD	Software Architecture & Design Specification
SRS	Software Requirements Specification
DRF	Django REST Framework
ORM	Object-Relational Mapping
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — a threat-modelling framework
JWT	JSON Web Token, used for stateless session authentication
RBAC	Role-Based Access Control

2. Document Overview

2.1 How to Use This Document

Section 3 (Architecture) should be read first by anyone evaluating the overall system structure, technology choices, and security posture. Section 4 (Design) should be read by developers implementing or reviewing specific modules, as it drills down into sequence-level interactions and the API contract. The Appendices provide supporting glossary, references, and tooling information.

2.2 Related Documents

- AttendEase Software Requirements Specification (SRS), v1.2, Team 3.
- AttendEase Project Synopsis, Team 3.

- AttendEase Software Test Plan (STP), Team 3.
- PES University SE Level 3 Mini-Project Guidelines and Deliverables Checklist.

3. Architecture

3.1 Goals and Constraints

Architectural goals:

- Clear separation of concerns between presentation, API, business logic, and data access, so that each of the four team-owned roles can develop and test their module independently.
- Maintainability and readability, given the project's academic timeline and four-person team.
- Security by design for authentication, role-based access, and sensitive attendance data.
- Testability — each service is unit-testable in isolation from the HTTP layer.

Constraints:

- Must be implemented in Python/Django within a single academic semester by a 4-member team.
- Must use Git/GitHub with a CI/CD pipeline (GitHub Actions) and Jira/GitHub Projects for sprint tracking, per course guidelines.
- Must run on commodity/free-tier hosting suitable for a student project (e.g., containerized deployment via Docker).

3.2 Stakeholders and Concerns

Stakeholder	Primary Concern
Students	Fast, accurate view of personal attendance; easy correction requests.
Faculty	Quick, reliable attendance marking; simple approval workflow for corrections.
Administrators	Accurate academic setup (courses, sections, faculty mapping); reliable reports.
Development Team (Team 3)	A modular design that maps cleanly onto the 4 owned roles and allows parallel development.
Course Evaluators	Clear, traceable architecture and design that satisfies IEEE-style documentation expectations.

3.3 Component (UML) Diagram

The diagram below shows AttendEase's layered component architecture: the Presentation Layer (role-specific web UIs), the API/Controller Layer (Django REST Framework endpoints), the Business Logic Layer (services, grouped by the four team-owned roles), the Data Access Layer (repositories over the Django ORM), the PostgreSQL database, and the external SMTP email service used for correction-request notifications.

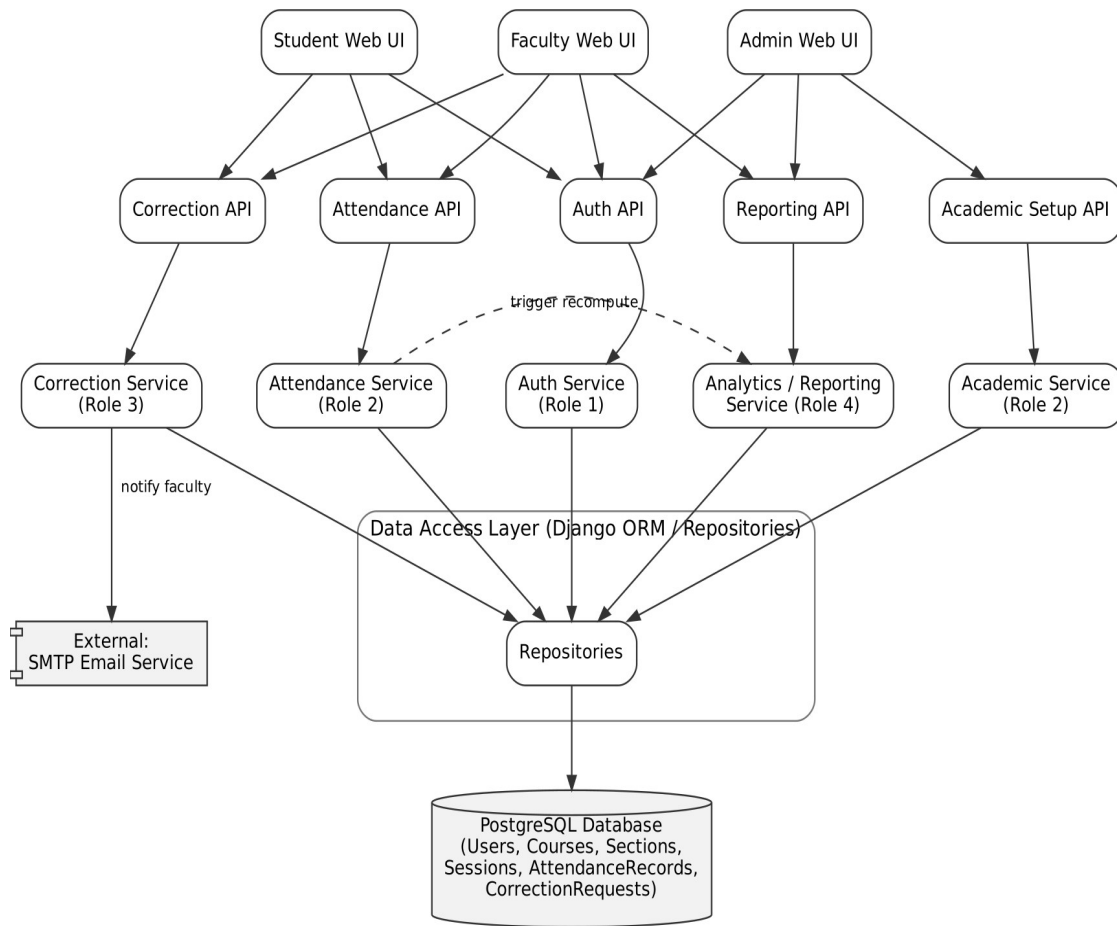


Figure 1: AttendEase component architecture (layered view).

3.4 Component Descriptions

Component	Responsibility
Student / Faculty / Admin Web UI	Role-specific browser-rendered views (Django templates / Bootstrap 5) that collect input and render data returned by the API layer.
Auth API & Auth Service	Registration, login, JWT/session issuance, and role-based access control (Role 1 — Atharva Ashish Vyas).
Academic Setup API & Academic Service	Course, section, and timetable management; faculty-to-section assignment (Role 2 — Arundhati K).
Attendance API & Attendance Service	Attendance session creation and per-student attendance recording (Role 2 — Arundhati K). Also enforces the duplicate-prevention and edit-window rules (FR-037, FR-038) at runtime, on behalf of Role 3's ownership of those requirements (see Correction API & Correction Service below).
Correction API & Correction Service	Raising, listing, and approving/rejecting attendance correction requests; owns the attendance-record duplicate-prevention and edit-window policy (FR-037, FR-038), which the Attendance Service enforces at runtime (Role 3 — Anshdeep Singh Sachdeva).
Reporting API & Analytics/Reporting Service	Attendance-shortage identification and report generation (Role 4 — Bhoomika Dayanand Jituri).

Component	Responsibility
Repositories (Data Access Layer)	Encapsulates all Django ORM queries, isolating business services from persistence details.
PostgreSQL Database	Durable storage for users, courses, sections, sessions, attendance records, and correction requests.
SMTP Email Service (external)	Sends correction-request notification emails to faculty.

3.5 Chosen Architecture Pattern and Rationale

AttendEase adopts a Layered (N-tier) architecture — Presentation, API/Controller, Business Logic (Service), and Data Access layers — implemented on top of Django and Django REST Framework.

Rationale:

- The layered pattern maps directly onto Django's own MVT conventions, minimizing framework friction for a student team.
- Each of the four team-owned roles (User Access & Student Portal; Academic & Attendance Setup; Attendance Updates & Corrections; Attendance Analytics & Reporting) corresponds to one vertical slice of API + Service, allowing the team to work largely in parallel with limited merge conflicts.
- A microservices architecture was considered and rejected: the added operational complexity (service discovery, inter-service networking, distributed transactions) is not justified for a single-database, single-deployment, academic-scale system with a 4-person team and a one-semester timeline.
- The Service layer isolates business rules (e.g., duplicate-attendance validation, shortage thresholds) from both the HTTP layer and the ORM, keeping logic unit-testable.

3.6 Technology Stack and Data Stores

Layer	Technology
Backend framework	Python 3.11, Django 4.x, Django REST Framework
Frontend	Django Templates, Bootstrap 5, vanilla JavaScript
Database	PostgreSQL 14+
Authentication	Django's auth system + JWT (djanoestframework-simplejwt) for API sessions
CI/CD	GitHub Actions (lint, test, build on every pull request) — chosen over Jenkins because it needs no self-hosted runner/infrastructure and integrates natively with the team's existing GitHub repository, which fits a 4-person student project better than operating a separate CI server.
Static Analysis	SonarQube (or an equivalent, e.g. SonarCloud's free tier) integrated into the GitHub Actions pipeline, to flag code smells, duplication, and common security issues on every pull request alongside the mandatory code review.
Containerization	Docker, docker-compose for local/staging environments
Project tracking	Jira / GitHub Projects (Kanban board, linked issues and PRs)
Email	SMTP (Django email backend) for correction-request notifications

3.7 Risks and Mitigations

Risk	Mitigation
Concurrent attendance edits by faculty causing data races	Database-level unique constraints on (session, student); service-layer validation before insert/update.
Scope creep given the 4-role split and semester timeline	Sprint-based plan of work with weekly Jira/GitHub Projects tracking and fixed sprint boundaries.
Team members unfamiliar with Django REST Framework	Shared code-review checklist and pairing during early sprints (Role: API integration & code review).
Loss of uncommitted work / merge conflicts	Mandatory feature-branch workflow with pull requests and CI checks before merge.
Sensitive attendance data exposure	Role-based access control and HTTPS-only deployment (see Section 3.9).

3.8 Traceability to Requirements

The table below maps architectural components to the functional, non-functional, and security requirements they implement, as defined in the AttendEase SRS Sections 5 and 6. FR-037 and FR-038 are listed under Correction API / Correction Service to match their ownership under Role 3 in the SRS (Section 5.3.1), even though the Attendance Service enforces them at runtime (see Section 3.4).

Component	SRS Requirement IDs
Auth API / Auth Service	FR-001–FR-005, FR-034; SEC-001, SEC-002
Academic Setup API / Academic Service	FR-006–FR-013
Attendance API / Attendance Service	FR-014–FR-019; NFR-001, NFR-002, NFR-004; SEC-004
Correction API / Correction Service	FR-024–FR-028, FR-037, FR-038; NFR-003; SEC-006
Reporting API / Analytics Service	FR-029–FR-033, FR-035, FR-036; NFR-001
Student/Faculty/Admin Web UI	FR-020–FR-023; NFR-007; and the presentation aspects of all FRs above

Cross-cutting requirements that apply across all components rather than a single one: NFR-005, NFR-006, and NFR-008 (maintainability, test coverage, and deployability apply system-wide); SEC-003 (server-side RBAC enforcement, applied per endpoint across every component) and SEC-005 (HTTPS, enforced at the infrastructure/reverse-proxy level).

3.9 Security Architecture

Security architecture is derived using a lightweight STRIDE threat-modelling pass over the system's trust boundaries (browser ↔ API, API ↔ database, API ↔ SMTP).

STRIDE Category	Representative Threat	Architectural Mitigation
Spoofing	An attacker impersonates a faculty or admin account.	Password hashing (PBKDF2/bcrypt via Django), JWT-based session tokens, mandatory login throttling/lockout after repeated failed attempts (SEC-002).
Tampering	A student modifies attendance data directly via API manipulation.	Server-side role-based authorization checks on every write endpoint; DB-level constraints.
Repudiation	A faculty member denies having approved/rejected a correction request.	Audit fields (acted_by, acted_at) stored on every CorrectionRequest; immutable status-change log.

STRIDE Category	Representative Threat	Architectural Mitigation
Information Disclosure	Attendance records of one section leak to unauthorized students.	Query-level scoping by authenticated user's role and enrolled sections; HTTPS in transit.
Denial of Service	Bulk/automated requests overwhelm the attendance-marking endpoint.	Rate limiting at the API gateway/reverse proxy layer; pagination on list endpoints.
Elevation of Privilege	A student account attempts to call admin-only academic-setup endpoints.	DRF permission classes enforcing RBAC per endpoint, verified in Section 6.3 (Security Objectives/Requirements) of the SRS.

4. Design

4.1 Design Overview

This section details two representative end-to-end workflows at the sequence level, the REST API contract, and cross-cutting design concerns (error handling, logging, UX). The two workflows were chosen because they represent the system's core write-paths and both faculty- and student-facing interactions: (1) Faculty starts a session and marks attendance, and (2) a student raises a correction request which a faculty member subsequently approves.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram 1 — Faculty Starts Session & Marks Attendance

Traces to FR-014, FR-015, and FR-016 (session creation, roster display, and per-student marking) and FR-037 (duplicate-record validation, shown as the "validate duplicates" step). Bulk-marking (FR-017), session closure (FR-018/FR-019), and the attendance-record edit-window flow (FR-038) are not depicted in this diagram; they follow the same Attendance Service but are separate flows described in SRS Sections 5.2.2 and 5.3.1.

Sequence Diagram 1: Faculty Starts Session & Marks Attendance

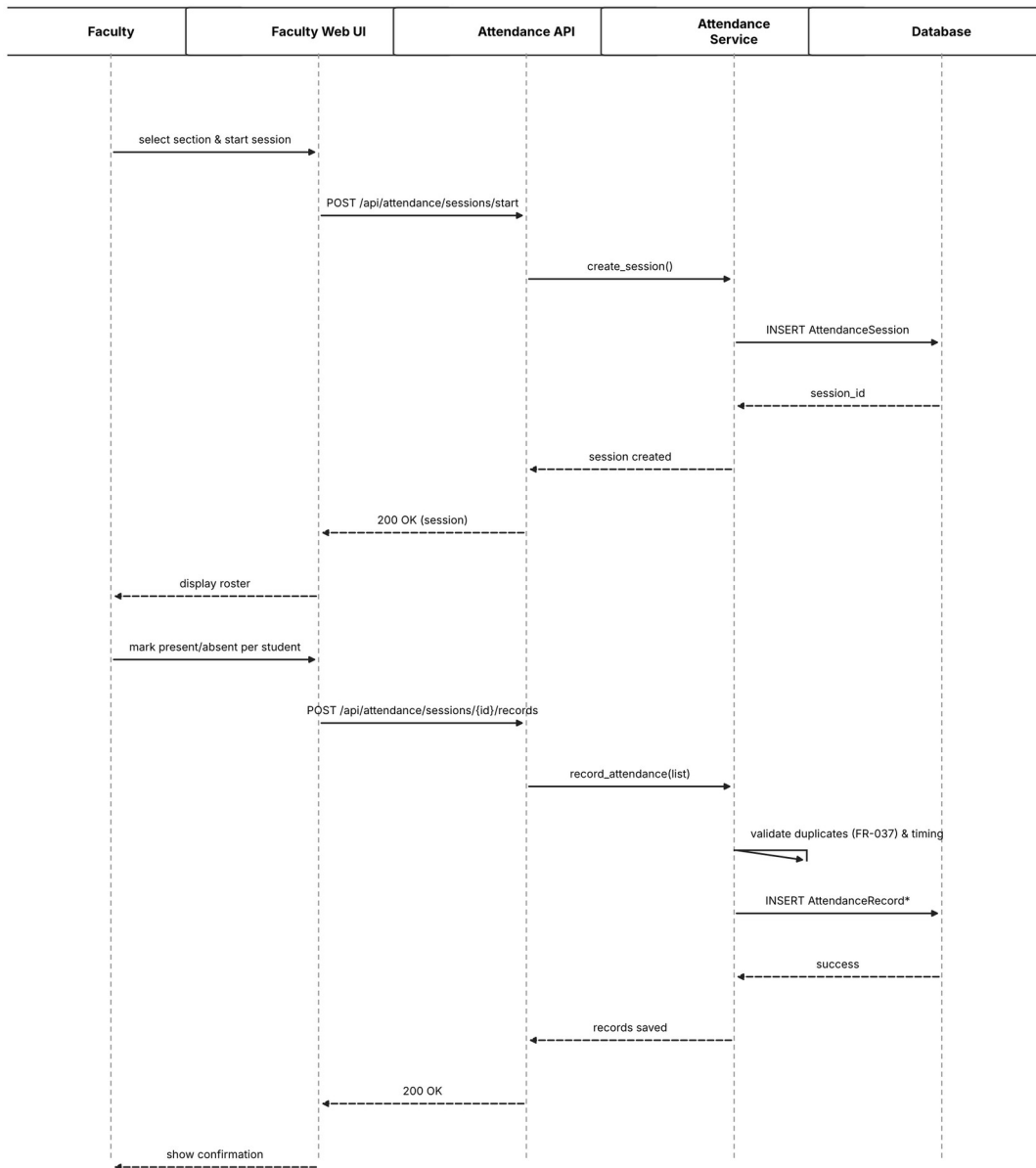


Figure 2: Faculty starts an attendance session and marks student attendance.

4.2.2 Sequence Diagram 2 — Student Raises Correction Request; Faculty Approves or Rejects

Traces to FR-024 through FR-028 (Attendance Correction Requests), including both the approval path (FR-027) and the rejection path (FR-028). The diagram separates the CorrectionRequest status update (with audit fields acted_by/acted_at, per SEC-006) from the downstream AttendanceRecord update, and shows the SMTP Service as an explicit external dependency rather than an internal step, consistent with the component diagram in Figure 1.

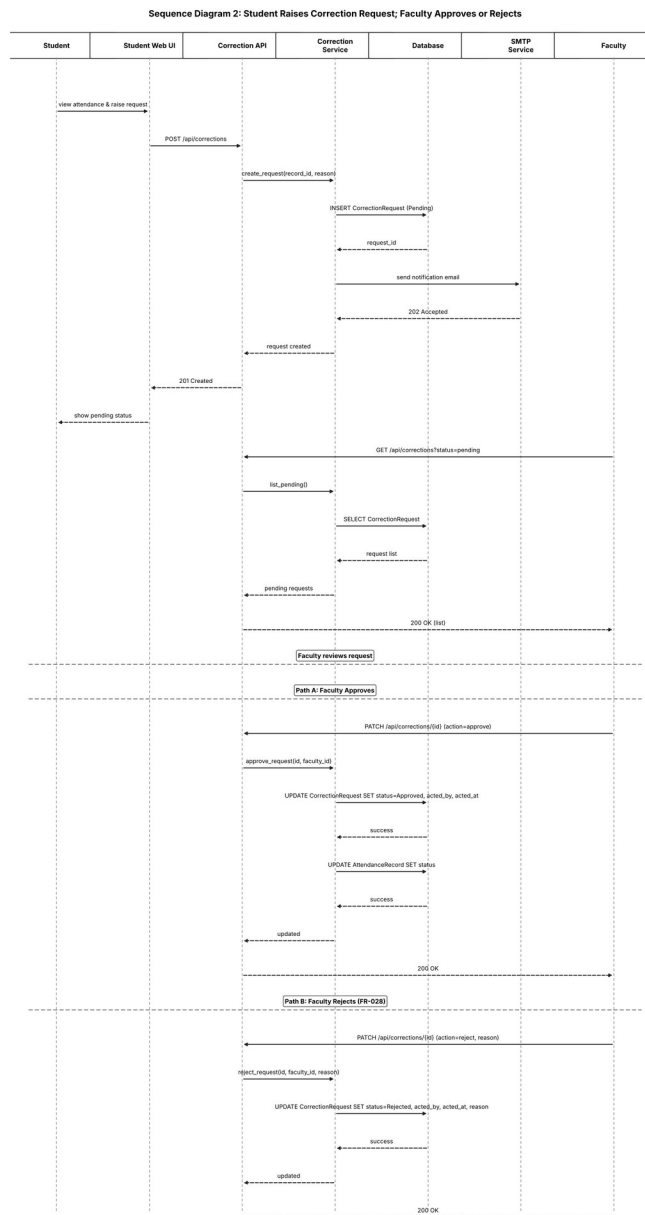


Figure 3: Student raises a correction request; faculty reviews and either approves or rejects it.

4.3 API Design

Representative REST endpoints exposed by the API/Controller layer (full contract to be maintained in the project's OpenAPI/Swagger specification during implementation). All endpoints other than registration and login require a valid JWT bearer token; the Auth/Role column states which role(s) may call the endpoint once authenticated.

Method & Path	Description	Auth / Role
POST /api/auth/register	Register a new user (student or faculty) with name, email, password, and role (FR-001).	Public
POST /api/auth/login	Authenticate and issue a JWT access/refresh token pair (FR-003).	Public
GET /api/academic/courses	List courses and sections.	Authenticated (any role)

Method & Path	Description	Auth / Role
POST /api/academic/assignments	Assign a faculty member to a course-section (FR-013).	Admin
POST /api/attendance/sessions/start	Start a new attendance session for a section (FR-014).	Faculty
POST /api/attendance/sessions/{id}/records	Submit attendance records for a session; rejects duplicates per FR-037.	Faculty
GET /api/attendance/dashboard	Retrieve the authenticated student's own attendance summary (FR-020).	Student
POST /api/corrections	Raise an attendance correction request against one of the student's own records (FR-024).	Student
PATCH /api/corrections/{id}	Approve (FR-027) or reject (FR-028) a correction request for a section the faculty member owns.	Faculty
GET /api/reports/defaulters	Generate an attendance-shortage/defaulters report (FR-031).	Faculty / Admin

Request/response contract, validation, and status codes — illustrated for the two endpoints exercised by Figures 2 and 3; the remaining endpoints follow the same conventions:

- POST /api/attendance/sessions/{id}/records — Request body: {"records": [{"student_id": int, "status": "present"|"absent"}]}. Validation: session must exist and be open; student_id must be enrolled in the session's section; no (session_id, student_id) pair may already have a record (FR-037). Responses: 200 OK with the saved records on success; 400 Bad Request for a malformed body; 403 Forbidden if the session does not belong to the calling faculty member; 404 Not Found for an unknown session; 409 Conflict for a duplicate record.
- PATCH /api/corrections/{id} — Request body: {"action": "approve"|"reject", "reason": string (required when action="reject")}. Validation: the request must currently be Pending; the calling faculty member must own the section the underlying AttendanceRecord belongs to; reason is required for a rejection (FR-028). Responses: 200 OK with the updated request on success (acted_by and acted_at are set server-side, never accepted from the client); 400 Bad Request if reason is missing on a rejection; 403 Forbidden if the faculty member does not own the section; 404 Not Found for an unknown request; 409 Conflict if the request is no longer Pending.

4.4 Error Handling, Logging and Monitoring

- All API errors return a consistent JSON envelope: {"error_code", "message", "details"} with appropriate HTTP status codes (400/401/403/404/409/500).
- Validation errors (e.g., duplicate attendance marking) are raised as 409 Conflict with a descriptive error_code so the frontend can show a precise message.
- Server-side exceptions are logged centrally (Python logging module → rotating file handler in development, structured JSON logs in production) with request correlation IDs.
- Critical failures (e.g., failed email notification on correction request) are retried once and logged as warnings rather than failing the parent request.
- CI pipeline runs automated tests on every pull request; failures block merge (see Test Plan, Section 5).

4.5 UX Design

- Student dashboard: single-page summary of attendance percentage per course, with a shortage warning banner when below the configured threshold.

- Faculty attendance-marking screen: roster-style checklist with bulk "mark all present" and per-student toggle, optimized for marking a full section in under two minutes.
- Correction-request flow: students submit a reason via a short form; faculty see a single pending-requests queue with one-click approve/reject.
- Consistent Bootstrap 5 component library across all three role-specific UIs for visual consistency and reduced frontend effort (owned by the UI/UX & frontend support secondary responsibility).

4.6 Open Issues and Next Steps

- Confirm production hosting target (e.g., free-tier PaaS) before the final deployment sprint.
- Decide the exact length of the attendance-record edit window (FR-038) in hours or days, and whether it is configurable per course like the shortage threshold (FR-035).
- Decide whether JWT refresh tokens are needed for this semester's scope, or whether a single short-lived access token is sufficient given the system's academic (non-continuous) usage pattern.
- Finalize the SonarQube (or equivalent) quality-gate thresholds to enforce in the CI pipeline before Sprint 2.

5. Appendices

5.1 Glossary

Term	Definition
Attendance Session	A single faculty-initiated instance of attendance-taking for one course-section on one date/period.
Defaulter	A student whose attendance percentage falls below the configured shortage threshold.
Correction Request	A student-raised request to amend an attendance record, subject to faculty approval.

5.2 References

- ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description.
- OWASP Top 10 (2021) — Web Application Security Risks.
- NIST SP 800-160 — Systems Security Engineering.
- AttendEase Software Requirements Specification (SRS), v1.2, Team 3.

5.3 Tools

- Graphviz — component/architecture diagram generation.
- Matplotlib (Python) — UML sequence diagram generation.
- draw.io / Swagger (planned) — ongoing diagram and API-contract maintenance during implementation.



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Test Plan (STP)

AttendEase – Student Attendance Management System Team 3 | Course UE24CS341A | PES University

Revision History

Version	Date	Description	Author(s)
1.0	03 October 2026	Initial Software Test Plan for AttendEase.	Team 3

Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026

1. Introduction

1.1 Purpose

This Software Test Plan (STP) defines the scope, approach, resources, and schedule of testing activities for AttendEase. It specifies which features will be tested, the testing levels and types to be applied, the environment and tools required, and the traceability between test cases and the requirements defined in the AttendEase Software Requirements Specification (SRS).

1.2 Scope

This plan covers functional testing of all 38 functional requirements (FR-001–FR-038), non-functional testing of performance, safety, and quality-attribute requirements (NFR-001–NFR-008), and security validation of SEC-001–SEC-006 and the Security Objectives defined in SRS Section 6.3. It applies to the AttendEase web application across its four team-owned feature areas.

1.3 References

- AttendEase Software Requirements Specification (SRS), v1.2, Team 3.
- AttendEase Software Architecture & Design Specification (SAD), v1.0, Team 3.
- PES University SE Level 3 Mini-Project Guidelines and Deliverables Checklist.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
STP	Software Test Plan
RTM	Requirements Traceability Matrix
FR / NFR	Functional Requirement / Non-Functional Requirement
SEC	Security Requirement (as numbered in the SRS)
CI	Continuous Integration

2. Test Items

- Authentication & Role-Based Access module (Role 1).
- Student Attendance Dashboard (Role 1).
- Student & Academic Management module (Role 2).
- Attendance Recording module (Role 2).
- Attendance Management & Correction Requests module (Role 3).
- Attendance Shortage Identification & Reports module (Role 4).
- REST API layer (all endpoints listed in the SAD, Section 4.3).
- Database schema and data-integrity constraints.

3. Features to be Tested

All functional requirements FR-001 through FR-038, all non-functional requirements NFR-001 through NFR-008, and all security requirements SEC-001 through SEC-006, as defined in the SRS, are in scope for testing. Representative coverage is detailed in the Test Case table in Section 13.

4. Features Not to be Tested

- Third-party infrastructure internals (e.g., the SMTP provider's own delivery guarantees, PostgreSQL's internal engine correctness) — these are assumed reliable and out of scope.
- Load testing beyond the expected single-department scale (not required for the mini-project scope; noted as a future-work item).
- Browser-compatibility testing beyond the latest stable versions of Chrome and Firefox.

5. Test Approach / Strategy

5.1 Security Validation

Security validation is performed as a dedicated testing activity, distinct from functional testing, covering both the Security Objectives and Security Requirements defined in SRS Section 6.3:

- Verify password hashing is applied (no plaintext passwords in the database) and that login endpoints reject invalid credentials correctly.
- Verify account lockout/throttling (SEC-002): after a configured number of consecutive failed login attempts on one account, further attempts are rejected until the cooldown/reset condition is met, even with a correct password.
- Verify role-based access control: attempt to call admin-only and faculty-only endpoints with a student token and confirm 403 Forbidden responses.
- Verify that a student cannot view or modify another student's attendance records via direct API calls (authorization bypass / IDOR check).
- Verify HTTPS is enforced in the staging/production configuration and that session tokens are not exposed in URLs or logs.
- Verify audit fields (acted_by, acted_at) are correctly populated on correction-request approvals/rejections (non-repudiation check).
- Basic input-fuzzing of form and API fields (SQL-injection and XSS payloads) against the registration, attendance, and correction-request endpoints.

5.2 Test Levels

Level	Description	Owner
Unit Testing	Individual service-layer functions tested in isolation (e.g., duplicate-attendance validation logic).	Module owner (per role)
Integration Testing	API endpoint tested against the service and database layers together.	Module owner + reviewer
System Testing	End-to-end workflows across UI → API → DB (e.g., full correction-request lifecycle).	Whole team
Acceptance Testing	Verification against SRS requirements prior to each sprint demo / final submission.	Whole team

5.3 Test Types

- Functional testing — validates each FR behaves as specified.
- Regression testing — re-run automatically via CI on every pull request to catch breakage from new changes.

- Performance testing — validates NFR-001/NFR-002 response-time targets under expected concurrent load.
- Usability testing — informal walkthroughs of the student/faculty UI with team members acting as test users.

5.4 Entry and Exit Criteria

Entry criteria:

- The feature's code has been merged to the main branch via a reviewed pull request and passes CI lint/build checks.

Exit criteria:

- All planned test cases for the feature have been executed with no open critical or high-severity defects.
- Test results have been recorded in the project's test-tracking sheet / GitHub Projects board.

6. Test Environment

Category	Detail
Hardware	Standard developer laptops; no specialized hardware required.
Software	Python 3.11, Django 4.x, PostgreSQL 14+, Docker, modern Chrome/Firefox browsers.
Tools	Django's built-in test framework / pytest-django, Postman or DRF's browsable API for manual API testing, GitHub Actions for CI execution, SonarQube (or equivalent) for static code analysis on every pull request.
Test Data	Seed fixtures representing at least 1 admin, 2 faculty, and 10 student accounts across 2 courses/sections.

7. Test Schedule

Milestone	Target Timeframe	Activity
Unit test scaffolding	Sprint 2	Each role sets up unit tests for their service layer as features are built.
Integration testing	Sprint 3–4	API-level tests added as endpoints stabilize.
System & security testing	Sprint 5	End-to-end and security validation across completed modules.
Acceptance / final regression	Sprint 6	Full regression pass before final submission and demo.

8. Test Deliverables

- This Software Test Plan document.
- Automated unit/integration test suite in the project repository, executed via CI.
- Requirements Traceability Matrix (Section 13).
- Test execution summary / defect log maintained in GitHub Projects.

9. Roles and Responsibilities

Role	Responsibility	Owner(s)
QA Lead	Coordinates the overall test plan, tracks coverage, and owns Sections 5.1 and 13 of this document.	Bhoomika Dayanand Jituri (Testing, CI/CD, Jira & documentation)
Test Engineer (per module)	Writes and executes unit/integration tests for their owned feature area.	Each team member, for their own role's modules
Developer	Fixes defects identified during testing; participates in code review.	All team members
Product Owner / Reviewer	Confirms acceptance criteria are met before sign-off.	Whole team, each sprint review

10. Risks and Mitigation

Risk	Mitigation
Limited time for dedicated security testing within the academic schedule.	Security validation (Section 5.1) is scheduled as its own milestone in Sprint 5, not left to the end.
Inconsistent test coverage across the four independently-owned modules.	QA Lead reviews coverage weekly against the RTM in Section 13.
Test data drift between developer environments.	Shared seed-fixture script checked into the repository and run via docker-compose.

11. Assumptions and Dependencies

- All four team members have local Docker and PostgreSQL environments matching the SAD's technology stack.
- The CI pipeline (GitHub Actions) is configured and operational from Sprint 2 onward.
- The SRS and SAD remain the authoritative source of requirements; any requirement change is reflected in the RTM.

12. Suspension and Resumption Criteria

Testing of a feature is suspended if a blocking defect prevents meaningful further test execution (e.g., the CI build is broken, or the database migrations fail). Testing resumes once the blocking defect is fixed and merged, and the CI pipeline passes again.

13. Test Case Management and Traceability

The 13 representative test cases below are traceable to SRS requirement IDs and cover functional (FR), non-functional (NFR), and security (SEC) requirements. The project's automated test suite will be developed during implementation and will expand this representative set into complete step-by-step cases for every FR/NFR/SEC, tracked against the RTM in Appendix C of the SRS.

Test Case ID	Description	Requirement(s)	Expected Result
TC-01	Register a new student account with valid details.	FR-001	Account created; confirmation shown; password stored hashed.
TC-02	Attempt login with an	SEC-002	Account is locked/throttled

Test Case ID	Description	Requirement(s)	Expected Result
	incorrect password repeatedly on one account, exceeding the configured failure threshold.		once the threshold is reached; further attempts are rejected until the cooldown/reset condition is met, even with the correct password.
TC-03	Admin assigns a faculty member to a course-section.	FR-013	Assignment saved; faculty sees the section on their dashboard.
TC-04	Faculty starts an attendance session and bulk-marks all students present with a single action.	FR-014, FR-016, FR-017	Session and attendance records created; roster reflects marked status for every student after the single bulk action.
TC-05	Faculty attempts to mark attendance twice for the same session/student.	FR-037	Second attempt rejected with a duplicate-record error (409).
TC-06	Student views their attendance dashboard.	FR-020, FR-021	Correct attendance percentage and per-course breakdown displayed.
TC-07	Student raises a correction request for a disputed record.	FR-024, FR-025	Request created with Pending status; faculty notified by email.
TC-08	Faculty approves a pending correction request.	FR-027, SEC-006	Attendance record updated; request marked Approved; audit fields (acted_by, acted_at) populated on the CorrectionRequest.
TC-09	Student account attempts to call an admin-only academic-setup endpoint.	SEC-003	403 Forbidden returned; action logged.
TC-10	Generate a defaulter report for a section below the shortage threshold.	FR-029, FR-031	Report lists exactly the students below threshold with correct percentages.
TC-11	Simulate 50 concurrent faculty/student requests while loading the attendance-marking roster for an 80-student section, and measure response time.	NFR-001, NFR-002	Roster loads within 3 seconds; 95% of API requests complete within 2 seconds under the 50-concurrent-user load.
TC-12	Run the service-layer automated test suite and measure statement/branch coverage.	NFR-006	Automated test coverage on service-layer business logic is at least 70%.
TC-13	A student authenticated as Student A attempts to view or act on Student B's attendance record via a direct API call with Student B's record ID.	SEC-004	403 Forbidden (or empty/filtered result) returned; Student A cannot read or modify a record outside their own enrolled sections.

14. Test Metrics and Reporting

- Requirement coverage: percentage of FR/NFR/SEC IDs with at least one passing test case, tracked weekly.
- Defect density: number of defects found per module, tracked via GitHub Projects issue labels.
- CI pass rate: percentage of pull requests passing all automated checks on first submission.
- A summary of these metrics is reviewed by the whole team at each sprint review.

15. Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026